

Statistical Learning 2



FH KUFSTEIN TIROL
University of Applied Sciences

General Information

- » Johannes Schwab, PhD
 - Email: johannes.schwab@fh-kufstein.ac.at
- » Consultation Hours:
 - on request via mail
- » Research:
 - applied mathematics (inverse problems)
 - medical imaging
 - machine learning



» Inverse Problems:

Given are data y and a model A for how these data emerge. The goal is now to find a solution x that explains the data.

Find x , such that $A(x) = y$

» Applications:

- Tomographic problems
- Parameter identification (e.g. in partial differential equations)
- Geophysical problems
- Imaging in general

» Machine Learning:

Learn a model from a given dataset, which is able to solve a specific problem in an „optimal“ way

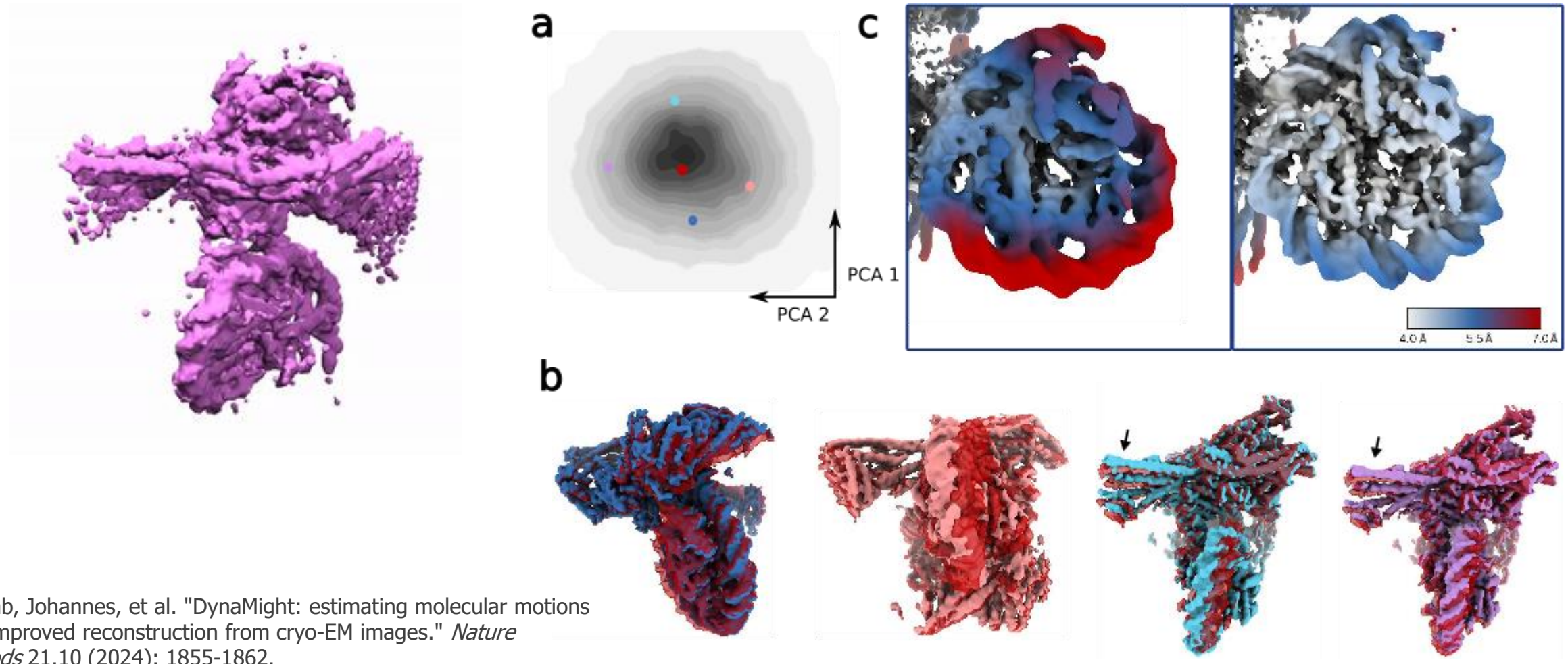
» Supervised and Unsupervised Learning:

- With paired data $[x, y]$ (supervised) learn a functional dependency ($f_{\Theta}(x) \approx y$).
- With unpaired data (unsupervised) detect patterns and analyse signal statistics.

» Applications:

- Image analysis (segmentation, classification, denoising, deblurring, inpainting,...)
- Prior extraction
- Biological applications
- ...

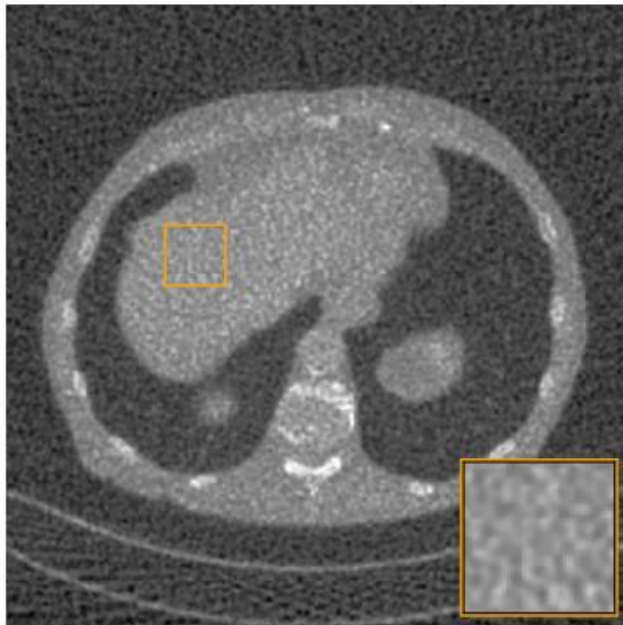
Example: Self Supervised Learning for estimating molecular motion



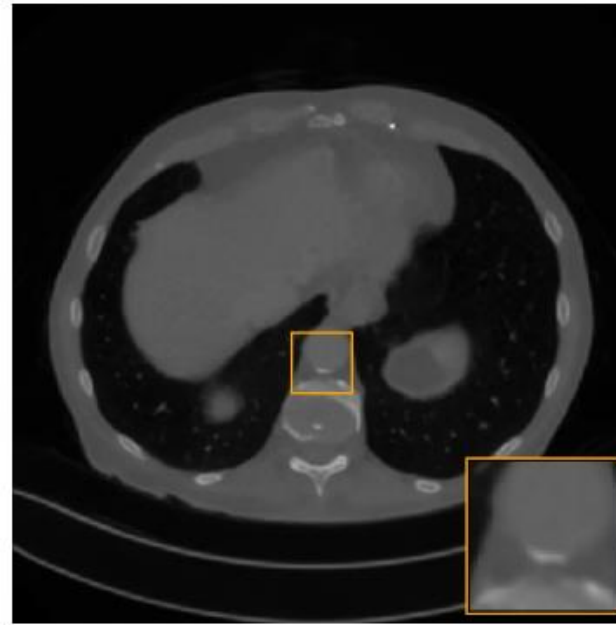
Schwab, Johannes, et al. "DynaMight: estimating molecular motions with improved reconstruction from cryo-EM images." *Nature Methods* 21.10 (2024): 1855-1862.

Deep Learning for Medical Imaging

- » Radiation reduction for CT imaging
- » Mitigating hallucinations in medical imaging with DL



without DL

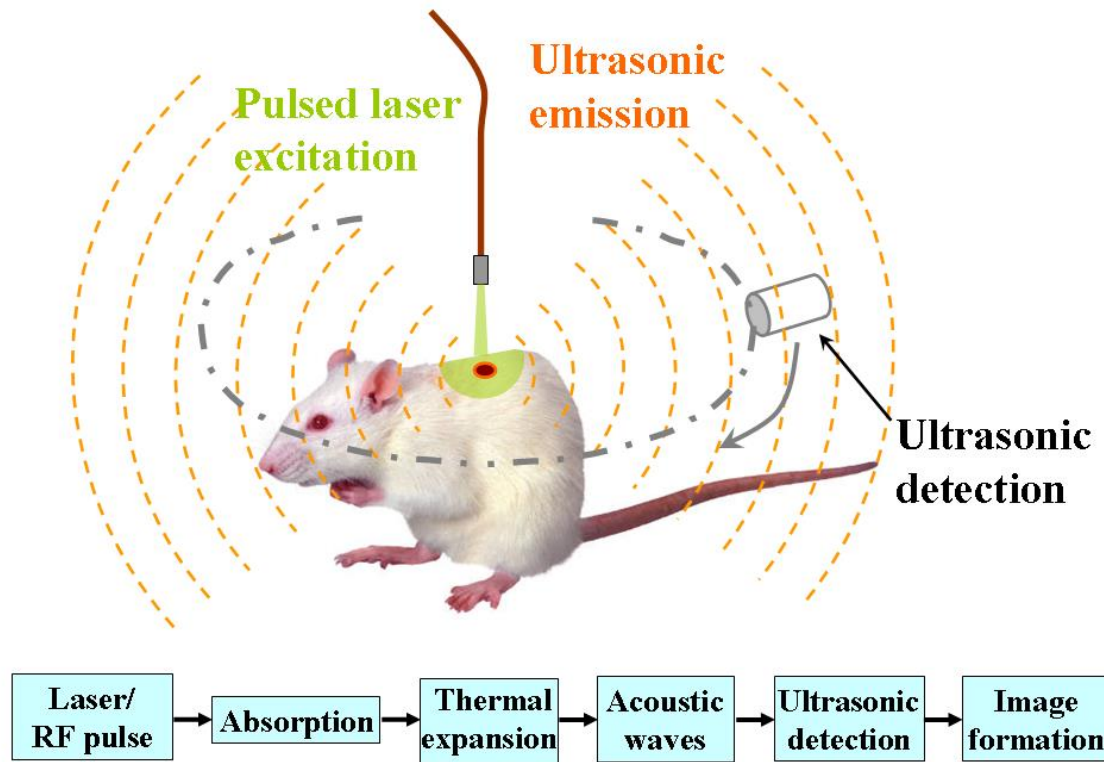


with DL

N. Gruber, J. Schwab, et al, "Noisier2Inverse: Self-Supervised Learning for Image Reconstruction With Correlated Noise," in *IEEE Access*, vol. 13, pp. 139445-139459, 2025

Schwab, Johannes, Stephan Antholzer, and Markus Haltmeier. "Deep null space learning for inverse problems: convergence analysis and rates." *Inverse Problems* 35.2 (2019)

Photoacoustic Tomography



Improving the reconstructions with Algorithmic advancements and machine learning.

- » Open to supervising master's theses, particularly in the following areas:
 - Image analysis
 - Image segmentation
 - Classification tasks
 - General machine learning
 - Mathematical modelling
 - Statistical analysis
 - Neural networks
 - Deep learning
 - Classical machine learning
 - Contrastive learning
 - ...

About you?

- » Brief description of your personal background, your activities and interests
- » You can also suggest a nickname that I can use throughout your studies

» Performance assessment

- **final exam:** The exam counts for 75% of the overall grade, and a positive exam grade (more than 60%) is **required** to pass the course. The exam will be a paper and pen exam without any tools.
- **contributions in the lecture and assignments:** The remaining **25%** consist of **in-lecture contributions** and possible assignments, and can be used to **improve (only) the exam grade**.

» Content and literature:

- Gareth James, Daniela Witten, Trevor Hastie and Robert Tibshirani: **“An Introduction to Statistical Learning”**
- Trevor Hastie, Robert Tibshirani and Jerome Friedman: **“The Elements of Statistical Learning”**
- Kevin P. Murphy: **“Probabilistic Machine Learning: An Introduction”**
- Ian Goodfellow, Yoshua Bengio and Aaron Courville: **“Deep Learning”**
- Christoph Molnar: **“Interpretable Machine Learning”**
- Solon Barocas, Moritz Hardt and Arvind Narayan: **“Fairness and Machine Learning”**

Organizational

Date and time	Tentative topic
Friday, 27.03. – 14:00-20:15 (online)	Recap, General Linear Models
Friday, 10.04. – 14:00-20:15 (online)	Bias-Variance Trade-Off and Regularization
Saturday, 18.04. – 08:45-15:30	Optimization
Saturday, 16.05. – 08:45-15:30	Resampling Methods and Model Assessment
Friday, 26.05. – 14:00-20:15	Probabilistic and Graphical Modeling
Saturday, 27.05. – 08:45-16:30	Fairness, Interpretability and Explainable AI
Friday, 03.07. – 12:00-14:00	Exam

- » Chapter 1: General Linear Models
- » Chapter 2: Bias-Variance Trade-Off and Regularization
- » Chapter 3: Optimization
- » Chapter 4: Resampling Methods and Model Assessment
- » Chapter 5: Probabilistic and Graphical Modeling
- » Chapter 6: Interpretability/Fairness/Responsible Modeling

General Linear Models



FH KUFSTEIN TIROL
University of Applied Sciences

Statistical Learning Problem

Suppose we observe data Y and p different variables, $x_1, x_2, \dots, x_p$. We assume that there is some relationship between y and $x = (x_1, x_2, \dots, x_p)$, which can be written in the very general form

$$Y = f(x) + \varepsilon.$$

Here f is some unknown function of $(x_1, x_2, \dots, x_p)$, and ε is a random error term independent of x with mean zero.

In this formulation f represents systematic information that x provides about y .

- » The goal of statistical learning algorithms is to estimate this unknown function f .

Why estimate f ?

There are two main reasons to estimate the function f :

- » Prediction (predict y from available x)
- » Inference (How to changes in x affect y)

- » In many cases the goal is to build a system that can make a prediction.
- » This system should take an input vector $x \in \mathbb{R}^n$ and predict the value of a scalar $y \in \mathbb{R}$.

$$f(x) \approx y$$

- » Examples:
 - Linear regression
 - Logistic regression (prediction of classes)
 - Neural Networks
 - Or in general: Supervised machine learning

- » In mathematical terms the goal is to build a model that takes input variables x and is able to predict y .

$$y = f(x)$$

- » This should hold for all observations $(x_1, y_1), (x_2, y_2), \dots, (x_K, y_K)$

$$y_j = f(x_j)$$

- » Note that we write x , even it has multiple dimension (e.g. x is a vector of dimension d) and use subscript to denote different samples from the dataset.

- » As the name already suggests, linear regression builds a model, where the output linearly depends on the input variables.

$$y_j = f(x_j) + \varepsilon_j = a_0 + a_1 x_j^1 + \dots + a_n x_j^n + \varepsilon_j$$

Here the superscripts are not powers but the components (dimensions) of the vector x_j

- » We can rewrite this as

$$y_j = a_0 + \sum_{i=1}^n a_i x_j^i$$

- » If we add a dimension to x , such that $x_0 = 1$, then $x \in \mathbb{R}^{n+1}$ we can write the previous expression as a vector multiplication

$$y_j = (x_j)^T a + \varepsilon_j$$

- » This gives us an equation for every observation, that is K equations with $n + 1$ unknowns.
- » We can simplify the notation to

$$y = Xa + \varepsilon$$

» In the expression

$$y = Xa + \varepsilon$$

the matrix $X \in \mathbb{R}^{K \times (n+1)}$ is sometimes called the design matrix or regressor matrix and a are the coefficients that we want to find such that we can separate the systematic component f from the random noise ε .

» In later slides a similar matrix is denoted by B .

» The matrix X has the following form:

$$X = \begin{pmatrix} 1 & x_1^1 & x_1^2 & \cdots & x_1^n \\ 1 & x_2^1 & x_2^2 & \cdots & x_2^n \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & x_K^1 & x_K^2 & \cdots & x_K^n \end{pmatrix}$$

Finding the Coefficients

- » To find the coefficients we want to minimize the error of the model f .
- » This means we want to find the vector a that minimizes the **objective function**

$$L(a) = ||y - Xa||^2$$

- » In order to find the minimizer we can differentiate with respect to a and obtain

$$\frac{\partial L}{\partial a}(a) = -2X^T y + 2X^T X a$$

Solving for the Coefficients

Setting the expression to zero gives (more details in the optimization chapter)

$$-2X^T y + 2X^T X a = 0$$

And solving for a gives and estimator

$$\hat{a} = (X^T X)^{-1} X^T y$$

- » **Linearity:** Note that nonlinear relationships can also be modeled. In the simplest case a nonlinear function can be estimated by transforming on of the input variables.
- » For this we introduce the concept of basis functions. In our simple example the model would look like this

$$y_j = a_0 \cdot 1 + a_1 B(x_j) + \varepsilon_j$$

where B is an arbitrary function. More basis functions are possible, and often : $B_0(x) = 1$

Example Basis Functions (Polynomial Basis 1D)

- » For a single input variable x we consider the basis functions
 - » $B_0(x) = 1$
 - » $B_1(x) = x$
 - » $B_2(x) = x^2$
 - » $B_3(x) = x^3$
 - » ...
- » The degree of the polynomial can be chosen.

Example Basis Functions (Multivariate polynomial)

- » For $x = (x_1, x_2)$ we can consider
 - » $B_0(x) = 1$
 - » $B_1(x) = x_1$
 - » $B_2(x) = x_2$
 - » $B_3(x) = x_1^2$
 - » $B_4(x) = x_1 x_2$
 - » $B_5(x) = x_2^2$
 - » ...
- » Again the degree of the polynomial can be chosen.

Example Basis Functions (Gaussian RBF basis)

- » We can choose centers $c_1, c_2, \dots, c_m$ and a width σ . Then consider the basis functions:
- » $B_0(x) = 1$
- » $B_1(x) = \exp(-(x - c_1)^2 / (2\sigma)^2)$
- » $B_2(x) = \exp(-(x - c_2)^2 / (2\sigma)^2)$
- » $B_3(x) = \exp(-(x - c_3)^2 / (2\sigma)^2)$
- » $B_4(x) = \exp(-(x - c_4)^2 / (2\sigma)^2)$
- » $B_5(x) = \exp(-(x - c_5)^2 / (2\sigma)^2)$
- » ...
- » Here we can choose the number of basis functions and the width of the Gaussians.

Example Basis Functions (other basis functions)

Other basis functions include:

- » Sigmoid basis: $B_k(x) = 1/(1 + \exp(-a(x - c_k)))$
- » Fourier basis:
 - $B_{2k-1}(x) = \sin(2\pi kx/T)$
 - $B_{2k}(x) = \cos(2\pi kx/T)$
- » Piecewise-constant basis: $B_k(x) = 1$, if $x \in I_k$, where I_k are intervals (bins)
- » Splines (next slides)

Solving for the coefficients

- » Solving for the coefficients works similar to linear regression, with the only difference, that the matrix is given by

$$B = \begin{pmatrix} B_0(x_1) & B_1(x_1) & B_2(x_1) & \cdots & B_n(x_1) \\ B_0(x_2) & B_1(x_2) & B_2(x_2) & \cdots & B_n(x_2) \\ B_0(x_3) & B_1(x_3) & B_2(x_3) & \cdots & B_n(x_3) \\ \vdots & & & & \\ B_0(x_K) & B_1(x_K) & B_2(x_K) & \cdots & B_n(x_K) \end{pmatrix}$$

Exercise

- » Create a jupyter notebook for the following exercise
- » Given are the following data points:

x	-3	-1	0	0.5	4
y	3	4	2	-1	2

- » Pick arbitrary basis functions and implement them as a functions in python
- » Compute the matrix B as well as $B^T B$ and $B^T y$
- » Solve for the coefficients a (use `np.linalg.solve`)
- » Define the Regression function and visualize it, together with the data points

Example Basis Functions: Splines

- » In many applications we need smooth functions
- » Piecewise polynomial functions
- » Degree of smoothness is defined by
 - Matching values at the knots (zero degree smoothness)
 - Matching derivatives at the knots (first degree smoothness)
 - Matching higher derivatives at the knots (higher degree smoothness)
- » 2D spline curves are parametric curves
$$S = \{f(t) \mid t \in [0, N]\}$$

Example Basis Functions: Splines

» The mathematical definition of a spline function $S(x)$ of degree d with knots at $t_1, t_2, \dots, t_k$ is:

$$S(x) = \begin{cases} P_1(x) & \text{if } x \leq t_1 \\ P_2(x) & \text{if } t_1 < x \leq t_2 \\ \vdots & \\ P_{k+1}(x) & \text{if } x > t_k \end{cases}$$

- » $S(x)$ is our spline function
- » d is the degree of the spline
- » $t_1, t_2, \dots, t_k$ are the knot points
- » $P_i(x)$ are polynomials of degree d or less. For example if $d = 3$ each $P_i(x)$ might look like

$$P_i(x) = a_i + b_i x + c_i x^2 + d_i x^3$$

Cubic Spline Regression

» The cubic spline regression model can be written as

$$y = a_0 + \sum_{j=1}^p a_j B_j(x) + \varepsilon$$

where:

- » y is the response variable (the value to be predicted)
- » a_0 is the intercept term
- » a_j the coefficient for the j –th spline basis function
- » $B_j(x)$ the j –th spline basis function
- » p the number of basis functions
- » x the input variable
- » ε the error term

- » The most elegant way to construct spline basis functions is through B-splines
- » B-splines are special functions that serve as the building blocks for constructing any spline function.
- » We can express any spline function as a linear combination of B-spline basis functions

$$S(x) = \sum_{j=1}^p a_j B_j(x)$$

Recursive Construction

- » B-splines are constructed recursively, starting from simple step functions and building up to more complex, smooth functions
- » We will construct them step by step

Zero-degree B-splines

- » The foundation of B-splines starts with zero-degree B-splines which are essentially step functions:

$$B_{i,0}(x) = \begin{cases} 1 & \text{if } t_i \leq x < t_{i+1} \\ 0 & \text{otherwise} \end{cases}$$

Here:

- » $B_{i,0}(x)$ represents the i –th zero-degree B-spline function. The subscript i tells us which knot interval this function is associated with.
- » $t_i \leq x < t_{i+1}$ is the interval where the function equals 1
- » Otherwise means that the function is 0 everywhere else.

Higher-degree B-splines

- » These are the building blocks for constructing smoother B-splines
- » To create smooth, higher-degree B-splines we use the Cox-de Boor recursion formula

$$B_{i,d}(x) = \frac{x - t_i}{t_{i+d} - t_i} B_{i,d-1}(x) + \frac{t_{i+d+1} - x}{t_{i+d+1} - t_{i+1}} B_{i+1,d-1}(x)$$

where

- » $B_{i,d}(x)$ is the i –th B-spline basis function of degree d evaluated at x
- » $B_{i,d-1}(x)$ is the i –th B-spline of degree $d - 1$ evaluated at x
- » $B_{i+1,d-1}(x)$ is the $(i + 1)$ –th B-spline of degree $d - 1$ evaluated at x
- » x is the input value
- » $t_i, t_{i+1}, \dots, t_{i+d+1}$ are the knot positions
- » i is the index identifying which basis function
- » D is the degree of the spline (0 for step functions, 1 for linear, 2 for quadratic,...)

Higher-degree B-splines

» The formula looks complex but we try to explain all the parts

- $\frac{x-t_i}{t_{i+d}-t_i}$ is a weighting factor that depends on where x is located relative to the knots. It's a linear function that:
 - » Equals 0 when $x = t_i$
 - » Equals 1 when $x = t_{i+d}$
 - » Varies linearly between 0 and 1 for values of x in between
- $\frac{t_{i+d+1}-x}{t_{i+d+1}-t_{i+1}}$ is another weighting factor that:
 - » Equals 1 when $x = t_{i+1}$
 - » Equals 0 when $x = t_{i+d+1}$
 - » Varies linearly between 1 and 0 for values of x in between

Higher-degree B-splines

- » This recursion formula combines two lower-degree B-splines using weighted averages. The weights ensure that:
 - The resulting function is smooth and continuous
 - The function has compact support (is non-zero only over a small interval)
 - The function maintains the desired continuity properties at the knots

Properties of B-splines

- » Compact support: Each B-spline $B_{i,d}(x)$ is non-zero only over a small interval, typically spanning $d + 1$ knot intervals. This means for a given x , only a few B-splines contribute to the final function value.
- » Partition of unity: The B-splines sum to 1 at every point:

$$\sum_{j=1}^p B_j(x) = 1$$

- » Non-negativity: Each B-spline satisfies $B_j(x) \geq 0$ for all x .
- » Continuity: B-splines automatically ensured the desired level of continuity at the knots. For example, cubic B-splines ensure that the function, its first derivative and second derivative are all continuous.

Exercise

» Consider the knot points:

$$t_1 = 0, t_2 = 1, t_3 = 2 \text{ and } t_4 = 3$$

- » Compute the B-spline basis functions of degree $d = 2$ using the definition of the zero-degree B-splines and the recursion formula.
- » Visualize the B-spline basis functions.

Matrix Formulation

- » We can express the spline regression model in matrix form. This will help us understand how to actually compute the spline regression using linear algebra.
- » The spline regression model can be written in matrix form as

$$y = \mathbf{B}a + \varepsilon$$

where:

- » y is the response vector (n observed values)
- » $\mathbf{B}$ is the spline basis matrix of dimension $n \times p$ (evaluated basis functions at all data points)
- » a is the coefficient vector of dimension p (weights for each basis function)
- » ε is the error vector (unexplained variation in responses)
- » n is the number of observations
- » P is the number of B-spline basis functions

Again we can solve the normal equation:

$$\hat{a} = (B^T B)^{-1} B^T y$$

To obtain the estimated coefficient vector $\hat{a}$.

Bias-Variance Trade-Off and Regularization



FH KUFSTEIN TIROL
University of Applied Sciences